

ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ № 12

Reverse Engineering

Թեմա՝ *Polymorphic / Metamorphic obfuscation, Anti-debugging,*

Packer և *Crackme* օրինակներ

Առարկա: Ծրագրային անվտանգություն

Խումբ: SS-355 (Լաբ.-1)

Ուսանող: Զոհրաբյան Հրայր

Outlook: hrayrzohrabyan.tt355@polytechnic.am

ՆԵՐԱԾՈՒԹՅՈՒՆ

Սույն լաբորատոր աշխատանքի նպատակն է ուսումնասիրել reverse engineering-ի հիմնական հասկացությունները, պարզել, թե ինչպես են կառուցվում packer-ները և crackme-ները, ինչպես նաև ծանոթանալ anti-debugging տեխնիկաներին:

Աշխատանքի առաջադրանքները

- Polymorphic / metamorphic obfuscation-ի ուսումնասիրություն
- Anti-debugging մեխանիզմներ (IsDebuggerPresent, ptrace)
- Պարզ executable packer-ի մշակում C++-ով
- Իրական crackme օրինակի կառուցում և reverse engineering

I. ՊԱՐՁ EXECUTABLE PACKER (C++)

Գաղափար

Packer-ը ծրագիր է, որը կատարում է հետևյալ քայլերը՝

1. Վերցնում է executable ֆայլը (.exe / binary)
2. Կոմպրեսացնում / կոդավորում է այն
3. Ստեղծում է նոր executable (stub + packed data)
4. Runtime-ում unpack է անում և կատարում

Պարզեցված տարբերակ — toy packer

Մեր toy packer-ը՝

- Ֆայլը կոդավորում է XOR ալգորիթմով
- Stub-ը decode է անում տվյալները և պահում որպես temp ֆայլ
- Ապա աշխատեցնում է unpacked ֆայլը

◆ PACKER (builder ծրագիր)

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <vector>
```

```
using namespace std;
```

```
vector<char> read_file(const string& path) {  
    ifstream file(path, ios::binary);  
    return vector<char>((istreambuf_iterator<char>(file)),  
                        istreambuf_iterator<char>());  
}
```

```
void write_file(const string& path, const vector<char>& data) {  
    ofstream file(path, ios::binary);  
    file.write(data.data(), data.size());  
}
```

```
void xor_encrypt(vector<char>& data, char key) {  
    for (auto& b : data) {  
        b ^= key;  
    }  
}
```

```

}

int main() {
    string input = "program.exe";
    string output = "packed.bin";
    char key = 0xAA;

    vector<char> data = read_file(input);

    xor_encrypt(data, key);

    write_file(output, data);

    cout << "Packed successfully!\n";
}

```

◆ STUB (unpacker + runner)

```

#include <iostream>
#include <fstream>
#include <vector>
#include <cstdlib>

using namespace std;

void xor_decrypt(vector<char>& data, char key) {
    for (auto& b : data) {
        b ^= key;
    }
}

```

```

}

int main() {
    ifstream file("packed.bin", ios::binary);
    vector<char> data((istreambuf_iterator<char>(file)),
                     istreambuf_iterator<char>());

    xor_decrypt(data, 0xAA);

    ofstream out("unpacked.exe", ios::binary);
    out.write(data.data(), data.size());

    system("unpacked.exe"); // run

    return 0;
}

```

Գործնական արդյունք

- Աշխատանք binary ֆայլերի հետ
- Պարզ encryption-ի կիրառում
- Runtime unpacking-ի իրականացում

Իրական *packer*-ները (օրինակ՝ *UPX*) շատ ավելի բարդ են. նրանք իրականացնում են *PE header manipulation*, *memory mapping* և այլ ցածր մակարդակի գործողություններ:

2. CRACKME ՕՐԻՆԱԿ (C++)

Գաղափար

Crackme-ն ծրագիր է, որը պահանջում է ճիշտ password / key: Նպատակը reverse engineering-ի միջոցով ճիշտ password-ի վերականգնումն է:

Πwpq Crackme (Obfuscated)

```
#include <iostream>

#include <string>

using namespace std;

// Obfuscated check

bool check(string input) {

    int key[] = {72, 29, 7, 0, 91};

    if (input.length() != 5)

        return false;

    for (int i = 0; i < 5; i++) {

        if ((input[i] ^ 0x55) != key[i])

            return false;

    }

    return true;
}

int main() {

    string pass;

    cout << "Enter password: ";

    cin >> pass;
```

```

    if (check(pass)) {

        cout << "Access Granted!\n";

    } else {

        cout << "Access Denied!\n";

    }

    return 0;

}

```

Ինչպես է աշխատում

Ստուգման հիմնական պայմանը՝

```
input[i] XOR 0x55 == key[i]
```

Իրական password-ը պահվում է կոդավորված տեսքով, ինչը խոչընդոտում է ուղղակի դիտմամբ ճանաչումը:

Reverse Engineering — լուծման գաղափար

XOR-ի առանձնահատկությունն այն է, որ այն հակադարձելի է: Հակադարձ գործողությունը՝

```
input[i] = key[i] XOR 0x55
```

Password-ի վերականգնում (helper ծրագիր)

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    int key[] = {72, 29, 7, 0, 91};

    for (int i = 0; i < 5; i++) {
        char c = key[i] ^ 0x55;
        cout << c;
    }
}
```

Արդյունք

Helper-ի կատարման արդյունքում էկրանին կտպվի իրական password-ը, որով հնարավոր կլինի «կոտրել» crackme-ն:

3. ԱՎԵԼԻ ԲԱՐԴ CRACKME ԳԱՂԱՓԱՐՆԵՐ

Crackme-ները կարող են զարգացվել հետևյալ մեթոդներով՝

◆ Anti-debugging

Ստուգում, թե արդյոք ծրագիրը գործարկված է debugger-ի ներքո.

```
#include <windows.h>

if (IsDebuggerPresent()) {
    exit(0);
}
```

◆ Timing attack detection

Debugger-ի առկայության դեպքում ծրագրի կատարման ժամանակը զգալիորեն դանդաղում է: Այս փաստը կարելի է օգտագործել՝ չափելով կոնկրետ հատվածների կատարման ժամանակը:

◆ Self-modifying code

Կողը runtime-ում փոփոխվում է ինքն իրենից, ինչը խոչընդոտում է ստատիկ վերլուծությունը:

ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

Կատարված լաբորատոր աշխատանքի արդյունքում՝

- ☒ Ուսումնասիրվել է packer-ների աշխատանքի սկզբունքը
- ☒ Մշակվել է պարզ XOR-հիմքով toy packer C++-ով
- ☒ Կառուցվել է obfuscated crackme և իրականացվել նրա reverse engineering-ը
- ☒ Ծանոթացել ենք anti-debugging հիմնական տեխնիկաներին

Ստացված գիտելիքները հիմք են հանդիսանում հետագայում ավելի բարդ reverse engineering խնդիրներ լուծելու համար, ինչպիսիք են PE format-ի մանիպուլյացիան, dynamic analysis-ը և malware-ի վերլուծությունը: